



Proyecto 3

Procesador Segmentado RV32I de 5 Etapas

Diseño de Sistemas Digitales – EL-3310

Profesor: Dr.-Ing. Jorge Castro-Godínez

Fabricio Mena Mejía
Julio David Quesada Hernández
Yair González Núñez

Índice

1. Descripción general	2
1.1. Cambios aplicados sobre el Proyecto 2	2
2. Módulo: pipeline_regs.sv	2
2.1. reg_IF_ID	3
2.2. reg_ID_EX	4
2.3. reg_EX_MEM	4
2.4. reg_MEM_WB	4
3. Módulo: hazard_unit.sv	4
3.1. Forwarding: resolución de hazards de datos sin penalización	5
3.2. Stall: resolución del hazard load-use	7
3.3. Flush: resolución del hazard de control	8
4. Modificaciones en top.sv	9
4.1. Etapa IF: PC con enable de stall	9
4.2. Etapa EX: muxes de forwarding	9
4.3. Cálculo del PC target	10
4.4. Etapa WB: BitSel y ALUResult[1:0] con registros locales	10
5. Correcciones aplicadas durante la integración	11
5.1. Bug: LUI escribía cero en el registro destino	11
6. Módulo: branch_predictor.sv	12
6.1. Estructura interna	12
6.2. Integración en top.sv	13
6.3. Impacto en la penalización de control	14
7. Programas de prueba	15
7.1. Programa 1: Manipulación Aritmética y Memoria Parcial	15
7.2. Programa 2: Conjetura de Collatz	15
8. Discusión de resultados	15

1. Descripción general

El Proyecto 3 transforma el procesador **uniciclo RV32I del Proyecto 2** en una arquitectura **segmentada de 5 etapas** (*pipeline*), conservando intactos todos los módulos funcionales originales (ALU, Unidad de Control, Banco de Registros, memorias, BitSelector, etc.) y añadiendo únicamente los componentes necesarios para la segmentación.

En el diseño uniciclo, cada instrucción ocupa un ciclo de reloj cuya duración está dictada por el *camino crítico* más largo (normalmente `lw`, que recorre $IF \rightarrow ID \rightarrow EX \rightarrow MEM \rightarrow WB$ de forma combinacional). Al segmentar el procesador, cada etapa opera en un ciclo más corto y cinco instrucciones distintas se solapan en cualquier instante, logrando en régimen estacionario **una instrucción completada por ciclo**.

La estructura del pipeline es:

$IF \rightarrow [IF/ID] \rightarrow ID \rightarrow [ID/EX] \rightarrow EX \rightarrow [EX/MEM] \rightarrow MEM \rightarrow [MEM/WB] \rightarrow WB$

El precio de esta ganancia de rendimiento son los *hazards* de datos y de control, que se resuelven mediante **forwarding**, **stall** y **flush**, implementados en los módulos `pipeline_regs.sv` y `hazard_unit.sv`.

1.1. Cambios aplicados sobre el Proyecto 2

Elemento	Acción
ALU, control_unit, main_decoder, alu_decoder	Sin cambios
extend, reg_file, single_port_ram, BitSelector	Sin cambios
instructionMemory, pc.sv, pc_increment.sv	Sin cambios
top.sv	Modificado: agrega hazard unit, registros de pipeline, muxes de forwarding y lógica de PCSrc en EX
pipeline_regs.sv	Nuevo: cuatro registros de segmentación IF/ID, ID/EX, EX/-MEM y MEM/WB
hazard_unit.sv	Nuevo: detección y resolución de hazards (forwarding, stall, flush)

Cuadro 1: Resumen de modificaciones respecto al Proyecto 2.

2. Módulo: pipeline_regs.sv

Este archivo contiene los cuatro registros de segmentación que dividen el datapath en etapas. Cada registro captura en el **flanco positivo del reloj** todas las señales de control y de datos que deben propagarse a la siguiente etapa.

El patrón fundamental de cada registro es:

```

1 always_ff @(posedge clk) begin
2     if (rst || flush)
3         <salidas> <= 0;           // insertar NOP / burbuja
4     else if (!stall)
5         <salidas> <= <entradas>; // captura normal
6     // Si stall=1 y no flush: retiene valor anterior
7 end

```

Listing 1: Patrón general de un registro de pipeline.

Las señales `stall` y `flush` provienen de la `hazard_unit` y se explican en detalle en la Sección 3.

2.1. reg_IF_ID

Separa las etapas **IF** e **ID**. Almacena la instrucción y el PC capturados al final de *Instruction Fetch*.

Puerto	Dir.	Descripción
<code>stall</code>	entrada	Congela el registro. Activado por hazard load-use.
<code>flush</code>	entrada	Inserta NOP. Activado cuando se toma un salto (<code>PCSrc_EX = 1</code>).
<code>PC_in</code> / <code>PCplus4_in</code> / <code>instr_in</code>	entrada	Valores capturados en IF.
<code>PC_out</code> / <code>PCplus4_out</code> / <code>instr_out</code>	salida	Valores visibles en ID.

Cuadro 2: Puertos de `reg_IF_ID`.

- **Stall:** el registro *no actualiza* sus salidas. La instrucción que estaba en él *permanece* un ciclo más, esperando a que el dato del `lw` esté disponible. El PC también se congela simultáneamente (ver Sección 4.1), de modo que el mismo par instrucción/PC se repite en la siguiente captura.
- **Flush:** el registro escribe `32'b0` en todos sus campos. En RISC-V, la instrucción `0x00000000` se decodifica como `addi x0, x0, 0` (NOP efectivo), de modo que ninguna operación incorrecta avanza por el pipeline.

¿Por qué congelar PC e IF/ID al mismo tiempo? Si solo se congela el PC pero no IF/ID, el registro IF/ID captura la instrucción *siguiente* y la instrucción que debía esperar se pierde. Si solo se congela IF/ID pero no el PC, el PC avanza y la instrucción que sigue en memoria nunca se fetcha. Ambos deben congelarse juntos para que el pipeline reanude correctamente.

2.2. reg_ID_EX

Separa las etapas **ID** y **EX**. Propaga todas las señales de control decodificadas y los operandos leídos del banco de registros.

Señales de control propagadas: RegWrite, ResultSrc[1:0], MemWriteEn, MemWrite[1:0], ALUSrc, ALUControl[3:0], BitSel[2:0], Sh, Jump, Branch.

Datos propagados: PC, PCplus4, rdata1, rdata2, imm, rd, rs1, rs2, opcode, funct3.

Este registro **no tiene señal stall**: cuando se detecta un hazard load-use, no se congela sino que se *vacía* (flush), insertando una burbuja. El stall ocurre aguas arriba (PC e IF/ID), mientras que ID/EX recibe un NOP para que la etapa EX no ejecute nada incorrecto ese ciclo.

2.3. reg_EX_MEM

Separa las etapas **EX** y **MEM**. Propaga el resultado de la ALU, el segundo operando (necesario para instrucciones **sw/sh/sb**) y las señales de control de memoria y writeback.

Este registro **no tiene señal flush ni stall**: para cuando una instrucción llega a esta etapa, ya se confirmó que debe ejecutarse. Los hazards ya fueron resueltos aguas arriba.

Incluye los puertos `imm_in/imm_out`, añadidos para corregir el bug de LUI descrito en la Sección 5.

2.4. reg_MEM_WB

Separa las etapas **MEM** y **WB**. Propaga el resultado de la ALU, el dato leído de memoria, PCplus4, el inmediato y las señales de control de escritura en registros (RegWrite, ResultSrc).

Al igual que `reg_EX_MEM`, no tiene flush ni stall.

3. Módulo: hazard_unit.sv

La hazard unit es un módulo **puramente combinacional** que *observa* el estado de los registros de pipeline y *decide* cómo corregir el flujo de instrucciones. No toca los datos directamente; solo genera las señales de control `ForwardA`, `ForwardB`, `stall` y `flush_ID_EX`.

```

1 module hazard_unit (
2     input  logic [4:0] ID_EX_rs1, ID_EX_rs2,    // fuentes en EX
3     input  logic      ID_EX_ResultSrc1,         // bit[0] de
        ResultSrc; 1=LOAD
4     input  logic [4:0] ID_EX_rd,                // destino en
        EX
5     input  logic [4:0] EX_MEM_rd,
6     input  logic      EX_MEM_RegWrite,
7     input  logic [4:0] MEM_WB_rd,
```

```

8     input  logic      MEM_WB_RegWrite,
9     input  logic [4:0] IF_ID_rs1, IF_ID_rs2,    // fuentes en
        ID (para stall)
10    input  logic      PCSrc,                    // salto
        tomado (desde EX)
11    output logic [1:0] ForwardA, ForwardB,
12    output logic      stall,
13    output logic      flush_ID_EX
14 );

```

Listing 2: Interfaz completa de hazard_unit.

3.1. Forwarding: resolución de hazards de datos sin penalización

Un hazard de datos tipo RAW (*Read After Write*) ocurre cuando una instrucción en EX necesita un dato que una instrucción anterior aún no ha escrito al banco de registros. En el pipeline de 5 etapas, ese dato puede estar en el registro EX/MEM (calculado hace 1 ciclo) o en MEM/WB (calculado hace 2 ciclos).

En lugar de detener el pipeline, se **cortocircuita** el dato directamente desde el registro de pipeline correspondiente hacia las entradas de la ALU. Los muxes 3-a-1 en la etapa EX realizan esta selección.

```

1 always_comb begin
2     // Forward A (entrada rs1 de la ALU)
3     if (EX_MEM_RegWrite && (EX_MEM_rd != 5'b0) && (EX_MEM_rd ==
        ID_EX_rs1))
4         ForwardA = 2'b10;    // desde EX/MEM (hazard EX, mas
        reciente)
5     else if (MEM_WB_RegWrite && (MEM_WB_rd != 5'b0) &&
        (MEM_WB_rd == ID_EX_rs1))
6         ForwardA = 2'b01;    // desde MEM/WB (hazard MEM)
7     else
8         ForwardA = 2'b00;    // banco de registros (sin hazard)
9
10    // Forward B (entrada rs2 de la ALU) -- misma logica
11    ...
12 end

```

Listing 3: Lógica de forwarding en hazard_unit.sv.

¿Por qué la prioridad EX/MEM > MEM/WB?

Considere la secuencia:

```

add x1, x2, x3    <- ciclo N    (escribe x1; llegara a EX/MEM en N+1)
add x1, x4, x5    <- ciclo N+1 (escribe x1; llegara a EX/MEM en N+2)
add x6, x1, x7    <- ciclo N+2 (necesita x1)

```

En el ciclo $N+2$, el valor *correcto* de $x1$ es el producido por la segunda `add` (que está en EX/MEM). El valor de la primera `add` todavía está en MEM/WB pero ya es obsoleto. Si se priorizara MEM/WB, se usaría el dato incorrecto. La prioridad EX/MEM > MEM/WB garantiza que siempre se use el dato **más reciente**.

¿Por qué `rd != 0`?

El registro $x0$ de RISC-V siempre vale cero y nunca se escribe efectivamente. Una instrucción con `rd = x0` no debe provocar forwarding, porque su “resultado” es irrelevante.

Conexión entre hazard_unit y los muxes en EX

En `top.sv`, `ForwardA` y `ForwardB` son señales `logic [1:0]` compartidas entre la salida de la hazard unit y los muxes:

```

1 // Mux 3-a-1 para la entrada A de la ALU
2 always_comb begin
3     case (ForwardA)
4         2'b00: ALUin0_pre = rdata1_EX;           // banco de
              registros
5         2'b10: ALUin0_pre = ALUResult_MEM;       // desde EX/MEM
6         2'b01: ALUin0_pre = writeback_data_WB;   // desde MEM/WB
7         default: ALUin0_pre = rdata1_EX;
8     endcase
9 end

```

Listing 4: Muxes de forwarding en la etapa EX (`top.sv`).

El sintetizador interpreta este `always_comb` con `case` como un multiplexor 3-a-1: `ForwardA` es la señal de selección, las tres asignaciones son las entradas de datos y `ALUin0_pre` es la salida. La hazard unit *conduce* la señal y el mux la *escucha*; no existe ninguna otra conexión explícita porque ambos viven en el mismo módulo `top`.

Forwarding para instrucciones STORE

Las instrucciones `sw/sh/sb` necesitan forwarding también sobre `rdata2`, que va a la memoria de datos y no a la ALU. Por eso en `top.sv` existe una señal separada `rdata2_fwd_EX` que aplica el mismo mux de `ForwardB` pero cuyo resultado se conecta a `reg_EX_MEM` en lugar de a la ALU:

```

1 always_comb begin
2     case (ForwardB)
3         2'b00: rdata2_fwd_EX = rdata2_EX;
4         2'b10: rdata2_fwd_EX = ALUResult_MEM;
5         2'b01: rdata2_fwd_EX = writeback_data_WB;
6         default: rdata2_fwd_EX = rdata2_EX;
7     endcase
8 end
9 // Este valor va a reg_EX_MEM, no a la ALU

```

Listing 5: Forwarding de `rdata2` para instrucciones `STORE`.

3.2. Stall: resolución del hazard load-use

El forwarding no puede resolver todos los hazards. Cuando una instrucción `lw` está en la etapa `EX`, su dato *todavía no existe*: recién se leerá de la memoria de datos en la etapa `MEM`, un ciclo después. La instrucción inmediatamente siguiente que depende de ese dato no puede recibir el valor correcto ni siquiera mediante forwarding.

La condición de detección es:

```

1 logic load_use_hazard;
2 assign load_use_hazard =
3     ID_EX_ResultSrc1                                //
4     && ((ID_EX_rd == IF_ID_rs1)                      // su destino
5       coincide con
6       || (ID_EX_rd == IF_ID_rs2))                  // rs1 o rs2
7       de la siguiente
8     && (ID_EX_rd != 5'b0);                          // no es x0

```

Listing 6: Detección de hazard load-use.

`ID_EX_ResultSrc1` es el bit 0 de `ResultSrc`. En este diseño, `ResultSrc = 2'b01` significa “el resultado proviene de memoria”, que es precisamente la definición de una instrucción `load`.

Cuando se detecta el hazard:

```

1 assign stall          = load_use_hazard;
2 assign flush_ID_EX    = load_use_hazard || PCSrc;

```

Listing 7: Activación de stall y flush por load-use.

El efecto sobre el pipeline durante un ciclo de stall es:

Etapa	Ciclo N	Ciclo N+1 (stall)	Ciclo N+2
IF	<code>instr_C</code>	<code>instr_C</code>	<code>instr_D</code>
IF/ID	<code>instr_B</code>	<code>instr_C</code>	<code>instr_C</code>
ID/EX	<code>instr_A</code>	<i>burbuja</i>	<code>instr_B</code>
EX/MEM	<code>lw</code>	<code>instr_A</code>	<i>burbuja</i>
MEM/WB	—	<code>lw</code>	<code>instr_A</code>

En el ciclo `N+2`, `lw` está en `MEM/WB` y el dato ya está disponible. El forwarding `MEM→WB` lo entrega a la instrucción que estaba esperando (`instr_B` en `ID/EX`) sin necesidad de ciclos adicionales.

3.3. Flush: resolución del hazard de control

Los branches y jumps se resuelven al **final de la etapa EX**, cuando la ALU calcula los flags de condición y el target de salto. Para ese momento, el procesador ya ha captado (*fetch*) dos instrucciones que *no debían ejecutarse*.

La señal PCSrc_EX se genera en `top.sv` replicando la lógica de BranchTaken con los flags reales de la ALU:

```

1 always_comb begin
2     BranchTaken_EX = 1'b0;
3     if (Branch_EX) begin
4         case (funct3_EX)
5             3'b000: BranchTaken_EX = ALUFlags_EX[0]; // beq:
                        Zero=1
6             3'b001: BranchTaken_EX = ~ALUFlags_EX[0]; // bne:
                        Zero=0
7             3'b100: BranchTaken_EX = ALUFlags_EX[1]; // blt:
                        Less=1
8             3'b101: BranchTaken_EX = ~ALUFlags_EX[1]; // bge:
                        ~Less
9             default: BranchTaken_EX = 1'b0;
10        endcase
11    end
12 end
13 assign PCSrc_EX = BranchTaken_EX | Jump_EX;

```

Listing 8: Cálculo de PCSrc en la etapa EX (`top.sv`).

¿Por qué no se usa el PCSrc de control_unit?

En el uniciclo, `control_unit` recibía los `ALUFlags` del *mismo* ciclo en que se ejecutaba la instrucción. En el pipeline, la unidad de control decodifica en la etapa ID, pero los flags reales de la ALU solo existen en la etapa EX. Si se usara el PCSrc de `control_unit` en ID, los flags serían de la instrucción *anterior*, produciendo decisiones de salto incorrectas. Por eso la lógica de BranchTaken se *replica* en EX usando `ALUFlags_EX`.

Cuando `PCSrc_EX = 1`:

- El PC se redirige a `PCTarget_EX` (el mux en IF lo selecciona).
- `reg_IF_ID` recibe `flush = PCSrc_EX` $\Rightarrow$ se convierte en NOP.
- `reg_ID_EX` recibe `flush_ID_EX = PCSrc` $\Rightarrow$ también se convierte en NOP.

Las dos instrucciones captadas incorrectamente son descartadas con una penalización fija de **2 ciclos** por salto tomado.

¿Por qué IF/ID y ID/EX reciben señales de flush distintas?

Registro	Señal de flush	Cuándo se activa
IF/ID	PCSrc_EX	Solo cuando hay salto tomado
ID/EX	flush_ID_EX = load_use_hazard PCSrc	Salto tomado o hazard load-use
EX/MEM	(ninguna)	No necesita flush
MEM/WB	(ninguna)	No necesita flush

IF/ID solo necesita limpiarse por salto (no por stall, ya que en ese caso se *congela*, no se vacía). ID/EX necesita limpiarse por ambas causas: en el stall recibe la burbuja para que la instrucción que esperaba no avance al EX con dato incorrecto; en el salto recibe la burbuja para descartar la instrucción que ya había pasado de IF a ID.

4. Modificaciones en top.sv

4.1. Etapa IF: PC con enable de stall

En el procesador uniciclo, el PC estaba encapsulado dentro de pc_increment. En el pipeline, el PC necesita una condición de congelamiento explícita:

```

1 always_ff @(posedge clk) begin
2     if (rst)
3         PC_reg <= 32'b0;
4     else if (!stall)           // stall=1: el PC no avanza
5         PC_reg <= PCnext;
6     // Si stall=1: PC_reg retiene su valor (flip-flop sin
7     // escritura)
8 end

```

Listing 9: Registro PC con enable de stall en top.sv.

El mux entre PCplus4 y el target de salto se instancia explícitamente antes del registro:

```

1 mux2to1_Nbits #(32) pc_mux_if (
2     .sel (PCSrc_EX),           // 0 = PC+4, 1 = target de salto
3     .a   (PCplus4_IF),
4     .b   (PCTarget_EX),
5     .y   (PCnext)
6 );

```

Listing 10: Mux de selección de PC siguiente.

4.2. Etapa EX: muxes de forwarding

Antes de las entradas de la ALU se insertan los muxes 3-a-1 controlados por ForwardA y ForwardB. La señal ALUin1_reg combina el forwarding con el mux ALUSrc (que selecciona entre registro e inmediato):

```

1 // 1. Forwarding (ForwardB selecciona la fuente del dato)
2 always_comb begin

```

```

3     case (ForwardB)
4         2'b00: ALUin1_pre = rdata2_EX;
5         2'b10: ALUin1_pre = ALUResult_MEM;
6         2'b01: ALUin1_pre = writeback_data_WB;
7         default: ALUin1_pre = rdata2_EX;
8     endcase
9 end
10
11 // 2. Mascara de shift (Sh_EX deja solo los 5 LSB)
12 assign ALUin1_sh = Sh_EX ? {27'b0, ALUin1_pre[4:0]} :
    ALUin1_pre;
13
14 // 3. ALUSrc: registro (con forwarding) o inmediato
15 assign ALUin1_reg = ALUSrc_EX ? imm_EX : ALUin1_sh;

```

Listing 11: Cadena de muxes hacia la entrada B de la ALU.

4.3. Cálculo del PC target

El destino del salto se calcula en EX según el tipo de instrucción:

```

1 assign PCTarget_EX = (opcode_EX == JALR_OP) ?
2     (ALUResult_EX & ~32'd1) : // JALR: rs1 +
    imm, bit 0 = 0
3     (PC_EX + imm_EX); // JAL /
    BRANCH: PC + offset

```

Listing 12: Cálculo de PCTarget en EX.

4.4. Etapa WB: BitSel y ALUResult[1:0] con registros locales

Para las instrucciones de carga parcial (1b, 1h, 1bu, 1hu), la etapa WB necesita saber el tipo de dato y la alineación en memoria. En lugar de agregar puertos extra al módulo reg_MEM_WB, se usan registros FF locales en top.sv:

```

1 always_ff @(posedge clk) begin
2     if (rst) begin
3         BitSel_WB         <= 0;
4         ALUResultLSB_WB <= 0;
5     end else begin
6         BitSel_WB         <= BitSel_MEM;
7         ALUResultLSB_WB <= ALUResult_MEM[1:0]; // bits de byte
            dentro de word
8     end
9 end

```

Listing 13: Registros locales para BitSel y alineación.

5. Correcciones aplicadas durante la integración

5.1. Bug: LUI escribía cero en el registro destino

Síntoma. La instrucción LUI `x2, 0x00012` dejaba `x2 = 0x00000000` en lugar de `0x00012000`.

Causa raíz. El registro `reg_EX_MEM` no tenía puertos `imm_in/imm_out`. La conexión original en `top.sv` era literalmente `.imm_in(32'b0)`, de modo que al llegar a la etapa MEM el inmediato se había perdido. La señal `ResultSrc = 2'b11` selecciona el inmediato como dato de escritura, pero recibía siempre cero.

Solución. Se extendió la cadena de propagación del inmediato:

1. Se añadieron los puertos `imm_in/imm_out` a `reg_EX_MEM` en `pipeline_regs.sv`.
2. Se conectó `imm_EX` $\rightarrow$ `imm_in` de `reg_EX_MEM`.
3. Se conectó `imm_MEM` $\rightarrow$ `imm_in` de `reg_MEM_WB`.

La cadena completa queda:

$$\text{imm_ID} \rightarrow \text{imm_EX} \rightarrow \text{imm_MEM} \rightarrow \text{imm_WB}$$

Con este fix, `ResultSrc = 2'b11` funciona correctamente y LUI escribe el inmediato desplazado en el registro destino.

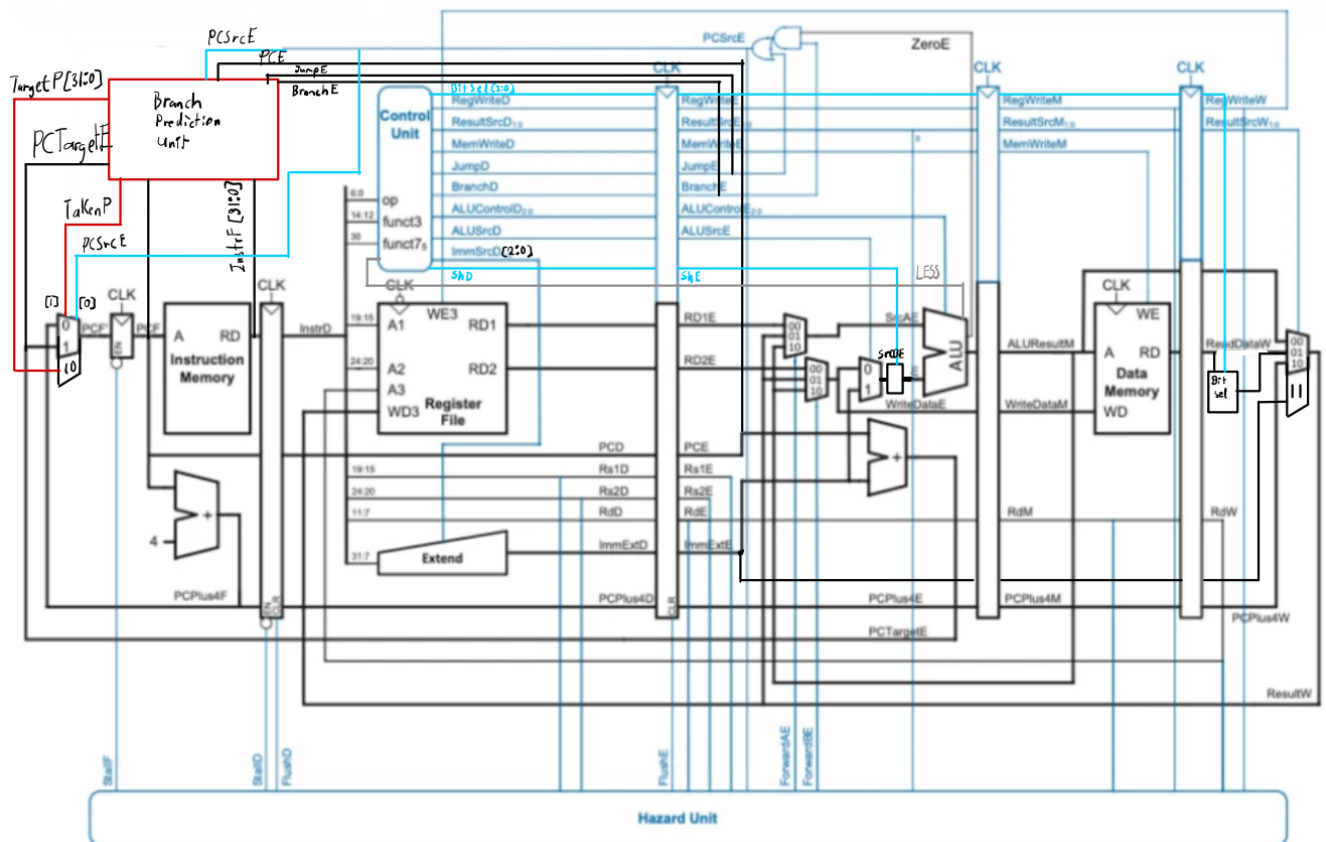


Figura 1: Microarquitectura final adaptada a todos los cambios realizados al procesador segmentado.

6. Módulo: branch_predictor.sv

El módulo `branch_predictor` implementa un predictor de saltos de 2 bits con tablas **BHT** y **BTB**, indexadas por `PC[7:2]` (64 entradas cada una). Su propósito es decidir en la etapa IF si un salto será tomado, redirigiendo el PC antes de que la instrucción llegue a EX, reduciendo la penalización de control de 2 ciclos a 0 cuando la predicción es correcta.

6.1. Estructura interna

Cada entrada de la **BHT** almacena un estado de 2 bits codificado como tipo enumerado:

```

1 typedef enum logic [1:0] {
2     STRONGLY_NOT_TAKEN = 2'b00,
3     NOT_TAKEN          = 2'b01,
4     TAKEN               = 2'b10,
5     STRONGLY_TAKEN     = 2'b11
6 } state_t;
7
8 state_t      bht [0:ENTRIES-1]; // Branch History Table
    (estado FSM)
9 logic [31:0] btb [0:ENTRIES-1]; // Branch Target Buffer
    (dirección destino)

```

Listing 14: Definición de la FSM en `branch_predictor.sv`.

El **BTB** guarda la dirección destino aprendida de cada salto. Ambas tablas son registradas (`always_ff`), por lo que Quartus las infiere como bloques de memoria con comportamiento de FSM sincrónica.

Las transiciones de estado se producen en el flanco positivo del reloj, habilitadas por `update_en` (activo cuando la instrucción en EX es `branch` o `jump`):

Las transiciones de estado se producen en el flanco positivo del reloj, habilitadas por `update_en`. La Tabla 3 resume las transiciones; el BTB se actualiza simultáneamente con `pc_target_ex`:

Estado actual	Tomado (<code>branch_taken_ex = 1</code>)	No tomado (<code>branch_taken_ex = 0</code>)
STRONGLY_NOT_TAKEN	NOT_TAKEN	STRONGLY_NOT_TAKEN
NOT_TAKEN	TAKEN	STRONGLY_NOT_TAKEN
TAKEN	STRONGLY_TAKEN	NOT_TAKEN
STRONGLY_TAKEN	STRONGLY_TAKEN	TAKEN

Cuadro 3: Transiciones de la FSM del contador saturante de 2 bits.

Los estados **STRONGLY_NOT_TAKEN** y **STRONGLY_TAKEN** actúan como extremos de saturación: un único resultado contrario no cambia la predicción, lo que otorga estabilidad ante saltos esporádicos en bucles de comportamiento dominante.

La predicción es combinacional: `bp_taken` se activa cuando `bht[idx_if][1] = 1` (estados `TAKEN` o `STRONGLY_TAKEN`) y la instrucción actual es un salto (`is_branch_if = 1`).

6.2. Integración en `top.sv`

La incorporación del predictor requirió cinco cambios puntuales en `top.sv`; ningún otro módulo fue modificado.

1. Detección de salto en IF

Se decodifica el opcode de `instr_IF` antes de que la instrucción avance a ID, para habilitar la consulta al predictor en el mismo ciclo de fetch:

```
1 assign is_branch_IF = (instr_IF[6:0] == BRANCH_OP) ||
2                      (instr_IF[6:0] == JAL_OP)      ||
3                      (instr_IF[6:0] == JALR_OP);
```

Listing 15: Detección rápida de instrucción de salto en IF.

2. Mux del PC ampliado a tres prioridades

El mux original de dos entradas (`PC+4` / `PCTarget_EX`) se reemplaza por lógica de tres prioridades:

```
1 always_comb begin
2     if (PCSrc_EX)          PCnext = PCTarget_EX;  // corrección
3     else if (bp_taken)    PCnext = bp_target;    // predicción
4     else                  PCnext = PCplus4_IF;   // flujo normal
5 end
```

Listing 16: Mux de PC con predictor.

La corrección desde EX tiene prioridad absoluta para garantizar la consistencia ante cualquier error de predicción.

3. Señal misprediction y propagación de `bp_taken`

Para detectar un error de predicción se compara el valor predicho (`bp_taken_EX`) con el resultado real (`PCSrc_EX`) una vez que el salto se resuelve en EX. Como `bp_taken` se genera en IF, debe propagarse dos ciclos hacia EX mediante registros de 1 bit:

```
1 // IF      ID
2 always_ff @(posedge clk) begin
3     if (rst || misprediction) bp_taken_ID <= 1'b0;
4     else if (!stall)          bp_taken_ID <= bp_taken;
5 end
```

```

6
7 // ID      EX
8 always_ff @(posedge clk) begin
9     if (rst || flush_ID_EX) bp_taken_EX <= 1'b0;
10    else                bp_taken_EX <= bp_taken_ID;
11 end
12
13 assign misprediction = (Branch_EX | Jump_EX) & (bp_taken_EX ^
    PCSrc_EX);

```

Listing 17: Propagación de bp_taken y cálculo de misprediction.

4. Flush condicional

Con el predictor, el flush de `reg_IF_ID` ya no se activa siempre que se toma un salto, sino *únicamente* cuando hay misprediction. Si la predicción fue correcta, las instrucciones en IF e ID ya corresponden al camino correcto y no deben descartarse:

Registro	Señal de flush (antes)	Señal de flush (ahora)
reg_IF_ID	PCSrc_EX	misprediction
reg_ID_EX	load_use_hazard PCSrc_EX	load_use_hazard misprediction

Cuadro 4: Cambio en las señales de flush al incorporar el predictor.

5. Actualización del predictor desde EX

La instancia `u_bp` recibe como retroalimentación las señales ya disponibles en EX, sin requerir señales nuevas en los registros de pipeline:

```

1 branch_predictor #(.ENTRIES(64)) u_bp (
2     .clk          (clk),
3     .rst          (rst),
4     .pc_if        (PC_IF),
5     .is_branch_if (is_branch_IF),
6     .bp_taken     (bp_taken),
7     .bp_target    (bp_target),
8     .update_en    (Branch_EX | Jump_EX),
9     .branch_taken_ex (PCSrc_EX),
10    .pc_ex         (PC_EX),
11    .pc_target_ex  (PCTarget_EX)
12 );

```

Listing 18: Conexión de la actualización del predictor.

6.3. Impacto en la penalización de control

En bucles con comportamiento predecible (la mayoría de los branches), el predictor aprende el patrón en 1–2 iteraciones y elimina la penalización en todas las siguientes, reduciendo

Caso	Ciclos de penalización
Sin predictor (salto tomado)	2
Predicción correcta	0
Misprediction	2

Cuadro 5: Penalización por hazard de control con y sin predictor.

el CPI efectivo de código con branches frecuentes.

7. Programas de prueba

Los mismos dos programas del Proyecto 2 validan el procesador segmentado. El testbench `tb_top.sv` se adapta con dos cambios:

- Se añaden **~5 ciclos de margen** sobre el número original de ciclos de espera, para compensar la latencia de llenado del pipeline (4 ciclos para la primera instrucción).
- Las verificaciones se toman en **flanco negativo** (`negedge clk`) para garantizar que la escritura del ciclo actual ya se estabilizó antes de leer el banco de registros.

7.1. Programa 1: Manipulación Aritmética y Memoria Parcial

Construye `0x12345678` mediante `LUI + ADDI + ORI + SLLI`, realiza stores y loads parciales (byte/halfword), operaciones de shift, comparaciones con signo/sin signo y ramas condicionales. Finaliza en loop infinito (`JAL`).

Ejercita: hazards RAW con forwarding, hazard load-use (stall de 1 ciclo), flush por branch y flush por JAL.

7.2. Programa 2: Conjetura de Collatz

Calcula los pasos de la conjetura de Collatz partiendo de $N = 7$, usando `JAL/JALR` para llamadas a subrutinas y branches condicionales para las iteraciones par/impar. Ejercita intensamente el flush por control hazard.

8. Discusión de resultados

- El pipeline de 5 etapas ejecuta correctamente las 34 instrucciones RV32I implementadas, produciendo los mismos resultados que el procesador uniciclo del Proyecto 2.
- El **forwarding** elimina la penalización para la gran mayoría de hazards de datos, sin detener el pipeline.
- El **hazard load-use** introduce exactamente **1 ciclo** de penalización, tras el cual el forwarding MEM→WB resuelve el conflicto de forma transparente.

- Los **saltos y branches** penalizan **2 ciclos** por instrucción tomada (las dos instrucciones fetched de más se descartan con flush).
- La **corrección del bug de LUI** fue esencial: sin propagar el inmediato a través de EX/MEM, todas las instrucciones que dependen de una carga de inmediato superior fallaban silenciosamente, escribiendo cero en el registro destino.
- Los programas Prog1 (aritmética/memoria) y Prog2 (Collatz) producen los mismos resultados que en el procesador uniciclo, validando la equivalencia funcional de ambas implementaciones.